CS 61A — Week 4

Recursion, mutual recursion, tree recursion (Lectures 9–10, Composing Programs 1.7)

§1 Facts used from Weeks 1–3

(W1) Calling a function creates a new frame, binds the parameters, and runs the body; the call expression's value is the return value.

(W2) A function's parent is the frame where it was defined; lookup goes local frame, parent, ..., global.

(W3) Digit tools: `n % 10` (last digit), `n // 10` (all but the last digit).

(W4) `and`/`or` short-circuit and return the last operand evaluated.

(W5) A helper function defined inside another function can use the outer function's names.

§2 This week

§2.1 Which frames an expression sees (Lecture 9)

You can tell where a name will be found without drawing a diagram:

- An expression outside every `def` and `lambda` is evaluated in the global frame only.
- An expression inside functions is evaluated in an environment with one frame for each enclosing `def`/`lambda`, from innermost to outermost, and then global.

```
def exp(x):
    def base(b):
        return pow(b, x)      # environment: a base frame, an exp frame, global
    return base
```

In `pow(b, x)`: `b` is found in the `base` frame, `x` in the `exp` frame, `pow` in global. This is how the lecture explained Midterm 1's "New Print": inside `f`, the name `print` is always found in a `new_print` frame.

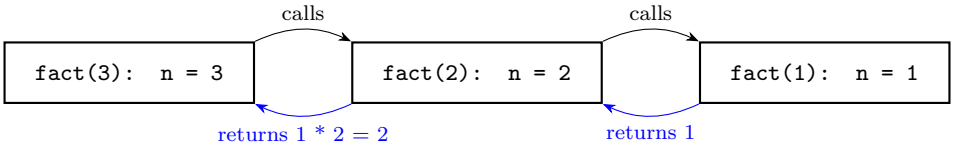
§2.2 Recursive functions (Lecture 9, CP 1.7)

The question. A function's body may call the function itself. Why does that not go on forever, and how can a function be used while it is still being written?

```
def fact(n):
    """Compute n factorial."""
    if n == 0 or n == 1:
        return 1                  # base case
    else:
        return fact(n - 1) * n    # recursive case
```

- A function is **recursive** if its body calls the function itself, directly or indirectly.
- **Base case**: an input simple enough to answer without a recursive call. It is tested first.
- **Recursive case**: the answer is built from the answer to a *smaller* instance of the same problem. $5! = 5 \cdot 4!$.
- Each recursive call must move toward a base case (here `n` decreases by 1).

> **The idea**
>
> Nothing new happens in the environment: each call gets its own frame with its own `n` (W1). `fact(3)` opens a frame with `n = 3` and must wait for `fact(2)`; that opens a second frame with `n = 2` and waits for `fact(1)`, which returns `1` at once. Then the waiting calls finish in reverse order: `1 * 2 = 2`, then `2 * 3 = 6`. The frames do not interfere, because each call looks up `n` in its own frame.



Three frames for one call to `fact(3)`. Calls go right; return values come back left.

How to know the recursive case is right (Lecture 10).

- **Tracing**: follow every call. Possible only for tiny inputs.
- **The recursive leap of faith** (induction): *assume* the function already works for every smaller input, and check that the recursive case then gives the right answer for this input. For `fact`: assume `fact(n - 1)` really is $(n-1)!$; then `fact(n - 1) * n` is $n!$. Together with a correct base case, that is a proof.

The leap of faith is functional abstraction applied to the function you are writing: use it by what it promises, not by how it runs.

How to get started on a recursive function (Lecture 10).

1. Start with the recursive case. Write down calls on smaller inputs you might make, what they return, and how those values could be combined into the answer.
2. Then find the base cases: follow the recursive calls and see where they end.
3. If the function is easy to write with `while`, write that first and convert (next subsection).

Worked exam question (Spring 2024 Midterm 1, Q4(e), from Lecture 10). A *dice integer* is a positive integer whose digits are all from 1 to 6. `streak(n)` returns whether `n` is a dice integer in which all digits are the same: `streak(22222)` and `streak(4)` are `True`; `streak(22322)`, `streak(99999)`, `streak(505)` are `False`.

```
def streak(n):
    return (n >= 1 and n <= 6) or (n > 9 and ______ and streak(______))
```

1. Smaller instance: `n // 10`, the number without its last digit. For 22222 that is 2222.
2. How does the answer for 2222 help? In a streak, everything except the last digit is a streak, *and* the last digit matches the digit before it.
3. "The last two digits match": `n % 10 == n // 10 % 10`.

4. Base case: a single digit from 1 to 6 (already given). A single digit outside 1–6, such as `9`, fails both sides and gives `False`; that is how `99999` is rejected.

```
    return (n >= 1 and n <= 6) or (n > 9 and n % 10 == n // 10 % 10 and streak(n // 10))
```

§2.3 Iteration and recursion (Lecture 9)

```
def fact_iter(n):
    result = 1
    while n > 0:
        result = result * n
        n -= 1
    return result
```

Converting a loop to recursion: the state of the loop becomes the arguments. The loop above has two changing names, `n` and the running total. Pass both:

```
def fact_k(n, k):
    """Compute n factorial times k."""
    if n == 0 or n == 1:
        return k
    else:
        return fact_k(n - 1, k * n)

def fact_tail(n):
    def f(n, k):
        if n == 0 or n == 1:
            return k
        else:
            return f(n - 1, k * n)
    return f(n, 1)
```

The idea

Two styles of recursion. In `fact`, work is done *after* the recursive call returns (`... * n`): the answer is assembled on the way back. In `fact_k`, work is done *before* the call (`k * n` is computed and passed along): the answer is carried forward in an argument and the base case returns it finished.

When the function you were asked for has too few parameters to carry the state, define an inner helper with the extra parameters and start it with the initial values: `return f(n, 1)`. The same recipe turns any `while` loop into recursion: one parameter per changing name, the loop condition becomes the base-case test, and the loop body ends with a recursive call on the updated values. (Lecture did this for the twenty-one game: the helper's arguments were the total and whose turn it is.)

§2.4 The order of events in a recursive call (CP 1.7)

Statements before the recursive call run on the way down; statements after it run on the way back, in reverse order.

```
def cascade(n):
    print(n)
    if n >= 10:
        cascade(n // 10)
        print(n)

>>> cascade(123)
123
12
1
12
123
```

§2.5 Helper functions and mutual recursion (Lecture 10)

```
def smallest_factor(n):
    """Return the smallest divisor of n above 1."""
    def smallest_divisor(k):
        "Return the smallest divisor of n above or equal to k."
        if n % k == 0:
            return k
        else:
            return smallest_divisor(k + 1)
    return smallest_divisor(2)
```

- The recursion is on a new quantity `k` that the original function does not have, so a helper is needed. The helper's docstring states exactly what it returns; that statement is what the leap of faith assumes.
- Here "smaller" means closer to the end: `k` increases toward a divisor (at the latest `k = n`).

Two functions are **mutually recursive** if each calls the other.

```
def unique_prime_factors(n):
    """Return the number of unique prime factors of n.
    >>> unique_prime_factors(576)    # 2*2*2*2*2*2*3*3
    2
    """
    k = smallest_factor(n)
    def no_k(n):
        "Return the number of unique prime factors of n other than k."
        if n == 1:
            return 0
        elif n % k != 0:
            return unique_prime_factors(n)
        else:
            return no_k(n // k)
    return 1 + no_k(n)
```

1. Find the smallest factor `k` (it is prime). It counts as one: `1 + ...`.
2. `no_k` divides `k` out as long as it divides (`no_k(n // k)`).
3. When `k` no longer divides what is left, the remaining number has only *other* prime factors: count them with `unique_prime_factors`. If nothing is left (`n == 1`) there are none.

For $72000 = 2^6 \cdot 3^2 \cdot 5^3$: count 1 for 2, leaving 1125; count 1 for 3, leaving 125; count 1 for 5, leaving 1. Answer 3.

§2.6 Tree recursion (Lecture 10, CP 1.7)

The question. How many ways are there to do something when each step offers a choice? Listing them is hopeless; counting them is a short function.

A function is **tree recursive** if one call makes more than one recursive call. The calls form a tree.

Lecture example: parking. `n` adjacent spots; each is empty (`.`), holds a motorcycle (`%`), or is half of a car (`<>`, two adjacent spots). Count the arrangements.

The idea

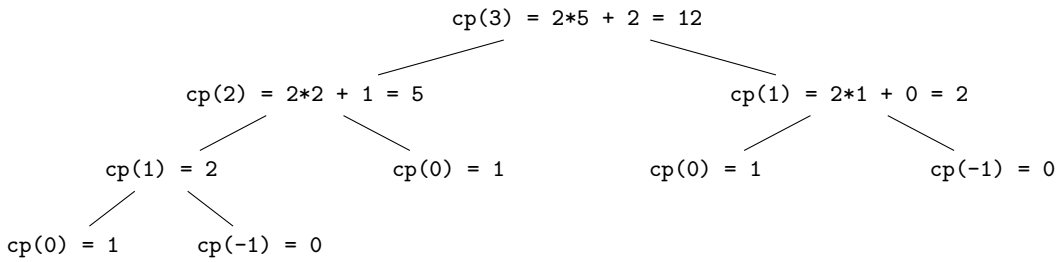
Decide what happens at the first spot, and count each possibility separately; the possibilities do not overlap and together cover everything.

- The first spot is empty: the remaining $n - 1$ spots can be filled in `count_park(n - 1)` ways.
- The first spot has a motorcycle: again `count_park(n - 1)` ways.
- A car starts at the first spot: it uses two spots, leaving `count_park(n - 2)` ways.

Base cases come from following the calls down: with 0 spots there is exactly one way (do nothing); with a negative number (a car that does not fit) there are none.

```
def count_park(n):
    if n < 0:
        return 0
    elif n == 0:
        return 1
    else:
        return 2 * count_park(n - 1) + count_park(n - 2)
```

`count_park(1) = 2`, `count_park(2) = 5`, `count_park(3) = 12`, `count_park(4) = 29`.



The tree of calls for `count_park(3)` (written `cp`). Every leaf is a base case.

Lecture example: counting partitions. `count_partitions(n, m)` is the number of ways to write `n` as a sum of positive integers, each at most `m`, in non-decreasing order. `count_partitions(6, 4)` is 9: $2+4$, $1+1+4$, $3+3$, $1+2+3$, $1+1+1+3$, $2+2+2$, $1+1+2+2$, $1+1+1+1+2$, $1+1+1+1+1+1$.

The idea

Choose a yes/no question that splits the partitions into two groups, each of which is a smaller instance of the same problem. The question: *is the largest allowed part, `m`, used at least once?*

- Yes: remove one `m`. What remains is a partition of `n - m` with parts up to `m` (another `m` may still be used). `count_partitions(n - m, m)`.
- No: every part is at most `m - 1`. `count_partitions(n, m - 1)`.

For $(6, 4)$: with a 4 there are `count_partitions(2, 4) = 2`; without, `count_partitions(6, 3) = 7`.

Base cases, by following the two kinds of call: `n` reaches `0` (a complete partition: one way); `n` goes negative (subtracted too much: no way); `m` reaches `0` while `n > 0` (no parts left: no way).

```
def count_partitions(n, m):
    if n == 0:
        return 1
    elif n < 0:
        return 0
    elif m == 0:
        return 0
    else:
        with_m = count_partitions(n - m, m)
        without_m = count_partitions(n, m - 1)
        return with_m + without_m
```

Fibonacci (CP 1.7). `fib(n) = fib(n - 1) + fib(n - 2)` with `fib(0) = 0`, `fib(1) = 1` is the simplest tree recursion. It recomputes the same values many times: the call tree for `fib(n)` has a number of calls that grows exponentially with `n`.

Shortcuts

- **Template for digit recursion**: base case a single digit (`n < 10`); recursive call on `n // 10`; combine with `n % 10`.
- **Counting problems**: split by the first choice (or by a yes/no question about one element), make one recursive call per option, add the results. "One way" when nothing is left; "zero ways" when you overshoot or run out of options.
- **Order of base cases matters**: test "success" (`n == 0`) before "failure" (`m == 0`), or a finished partition is counted as a failure.
- **Need extra state?** Write a helper with more parameters and call it with the starting values.
- **One-line recursive booleans**: `base or (guard and local_check and f(smaller))`, as in `streak`.
- **Print before or after?** Lines printed before the recursive call appear in call order; lines printed after appear in reverse.
- **Trust the docstring**: when reading or filling in a recursive call, replace it mentally by what the docstring says it returns.

Traps

- A missing or unreachable base case gives `RecursionError`: for example `fact(-1)`, or recursing on `n` instead of `n - 1`.
- Forgetting `return` in the recursive case: the call happens, its value is dropped, and the function returns `None`.
- In tree recursion, the options must not overlap and must cover every case, or you count twice or miss some.
- `count_partitions(n - m, m)`, not `(n - m, m - 1)`: after using one `m` you may use `m` again.

- A helper's `n` can hide the outer function's `n` (as in `no_k(n)`); check which frame each name belongs to.

§3 Problem set

All problems here are original; the exam question for this week is the worked `streak` example above. Programming exercises are in `exercises/`.

P1 (trace). What is displayed?

```
def f(n):
    if n == 0:
        print('base')
        return 0
    print('down', n)
    result = f(n - 1) + n
    print('up', n, result)
    return result

print(f(2))
```

P2 (find the bug). Each function is meant to return the sum of the digits of a positive integer. Say what goes wrong with each.

```
def sum_a(n):
    return n % 10 + sum_a(n // 10)

def sum_b(n):
    if n < 10:
        return n
    sum_b(n // 10) + n % 10

def sum_c(n):
    if n < 10:
        return n
    return sum_c(n) + n % 10
```

P3 (leap of faith). Write `count_up(n)`, which prints the integers from `1` to `n` in increasing order, one per line, using recursion and no loop. Explain the order of the two statements in the recursive case.

P4 (iteration to recursion). Convert to a recursive function with a helper and no loop:

```
def reverse(n):
    result = 0
    while n > 0:
        result = result * 10 + n % 10
        n = n // 10
    return result
```

P5 (tree recursion by hand). A staircase has `n` steps; each move climbs 1 or 3 steps. Write `climb(n)`, the number of different sequences of moves that reach the top exactly. Give `climb(5)` by drawing the tree or tabulating `climb(0)`, ..., `climb(5)`.

P6 (design the split). `count_strings(n)` is the number of strings of length `n` made of the letters `a` and `b` that never contain `bb`. Find the recursive case by asking what the first letter is, state the base cases, and compute `count_strings(4)`.